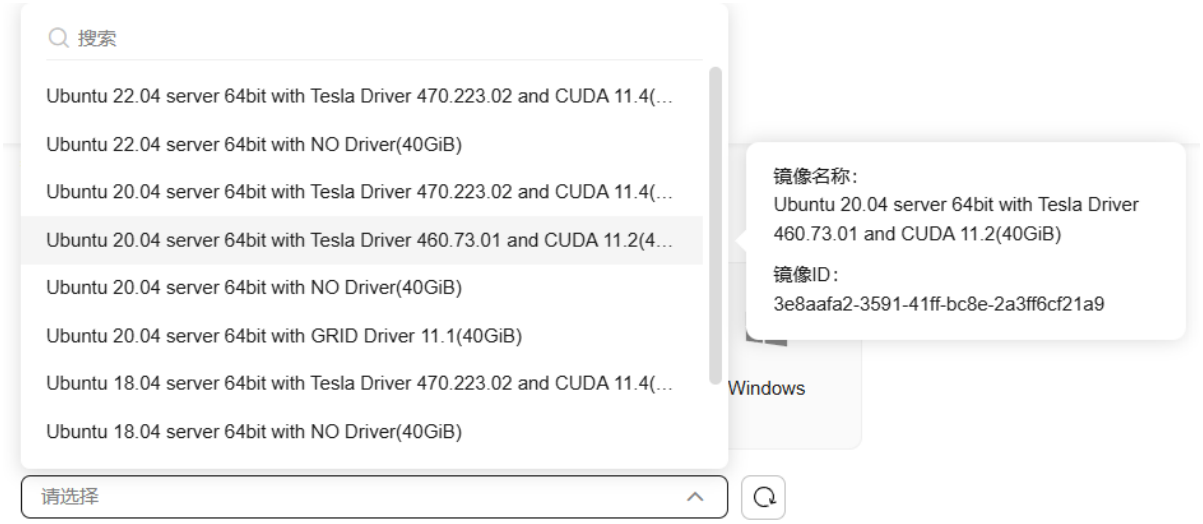
修复创建GPU服务器后无法导入依赖包的问题

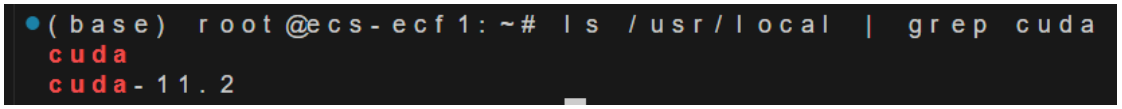
注：使用CPU服务器或自己电脑来做项目的同学不需要看这份文档。

详细步骤

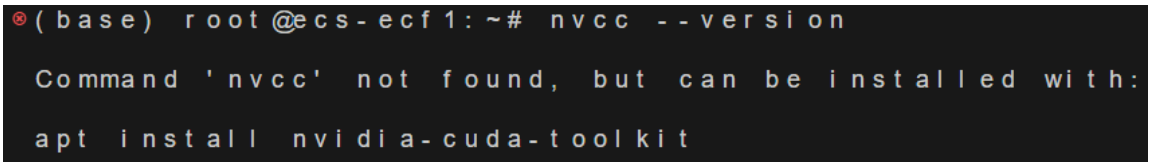
在华为云中创建含GPU的服务器时，根据课程内容，镜像选择为下图所示的选项。

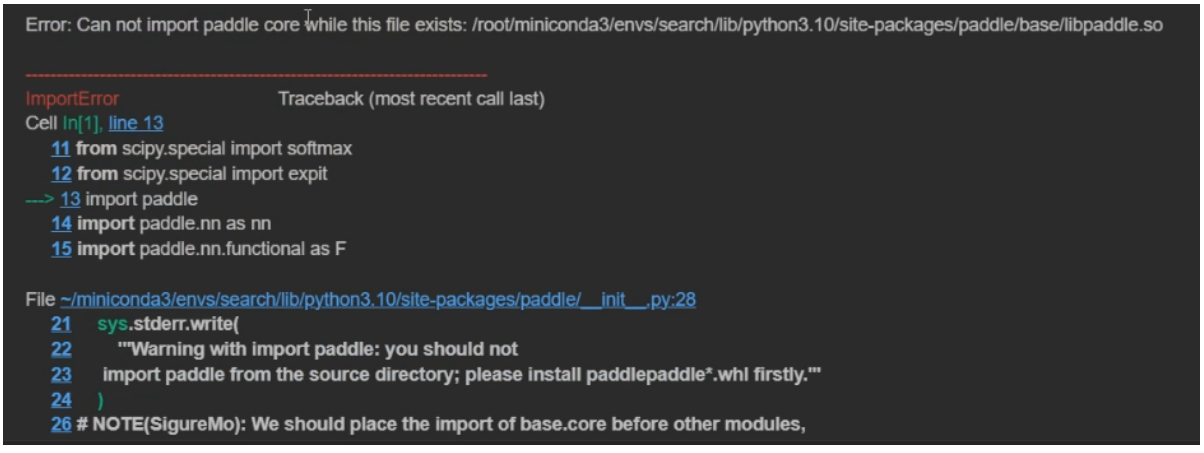


在服务器创建完成后，服务器中已经安装了cuda，运行命令“**ls /usr/local | grep cuda**”就可以看到cuda安装的位置。



但由于此时cuda未配置到环境变量中，因此使用“nvcc --version”命令会报错，而且在运行课程文件中的代码时，也会出现无法导入依赖包的问题，如下图所示。





```
   26 # NOTE(SigureMo): We should place the import of base.core before other modules,
   27 # because there are some initialization codes in base/core/__init__.py.
---> 28 from .base import core  # noqa: F401
   29 from .batch import batch
   31 # Do the *DUPLICATED* monkey-patch for the tensor object.
   32 # We need remove the duplicated code here once we fix
   33 # the illogical implement in the monkey-patch methods later.

File ~/miniconda3/envs/search/lib/python3.10/site-packages/paddle/base/__init__.py:36
   33     except Exception as e:
   34         raise e
...
  274         "reinstall paddlepaddle by 'python -m pip install --force-reinstall paddlepaddle-gpu[==version]' "
  275         "to get better performance.\n"
  276     )

ImportError: libcudart.so.11.0: cannot open shared object file: No such file or directory
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

为了解决这个问题，先正常执行完课程资料中“环境配置命令-服务器.txt“中的所有命令，

环境配置命令-服务器.txt

文件　编辑　查看

```
mkdir miniconda

cd miniconda

wget https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3-latest-Linux-x86_64.sh

conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/

conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/

conda config --set show_channel_urls yes

pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple


创建conda环境：

conda create --name search python==3.10


配置环境（安装依赖包）：

pip install numpy

GPU版本（根据cuda版本选择）：
python -m pip install paddlepaddle-gpu==2.6.2.post112 -i https://www.paddlepaddle.org.cn/packages/stable/cu112/
CPU版本：
pip install paddlepaddle==2.6.2 -i https://www.paddlepaddle.org.cn/packages/stable/cpu/

pip install paddlenlp

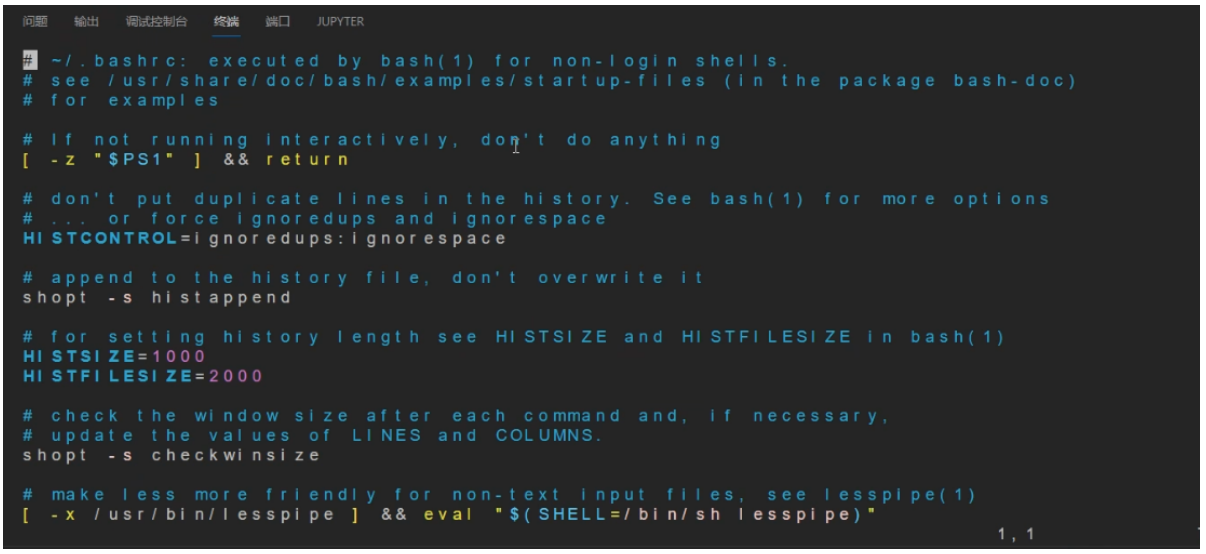
pip install hnswlib

pip install ipykernel
```

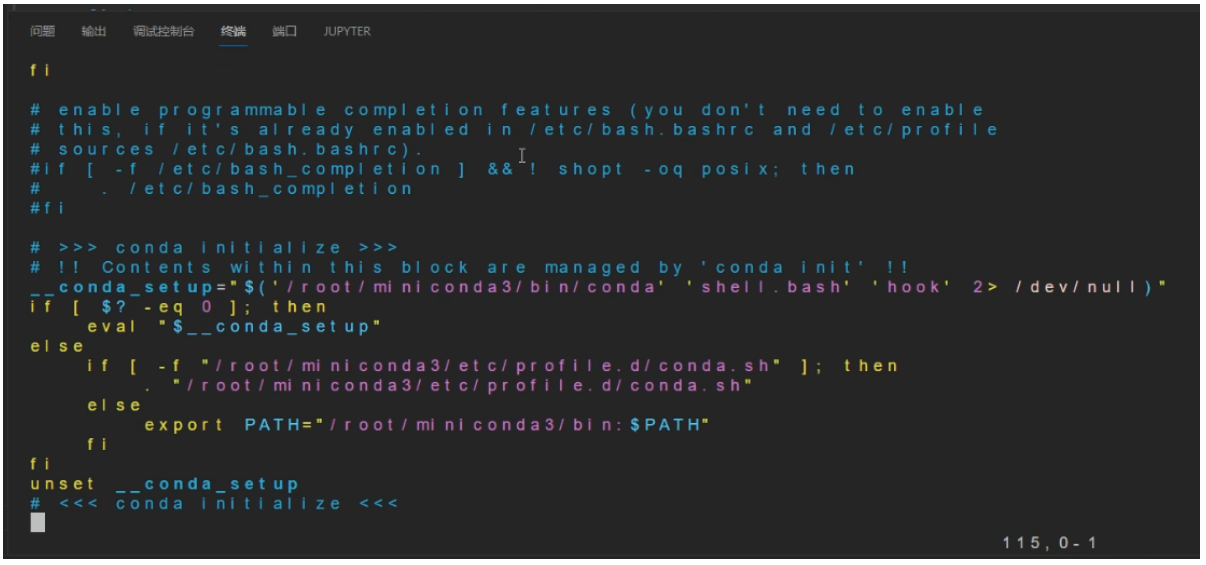
然后在服务器终端中输入“**vim ~/.bashrc**”命令，如下图所示，这个命令的作用是编辑.bashrc文件，配置环境变量。

```
(base) root@ecs-ecf1:~# vim ~/.bashrc
```

点击回车后，会出现.bashrc文件的内容，如下图所示。



然后按住键盘上的“↓”键，将光标移动到最后一行；然后按住键盘上的“→”键，将光标移动到最后一个字符后面。移动完成后如下图所示。

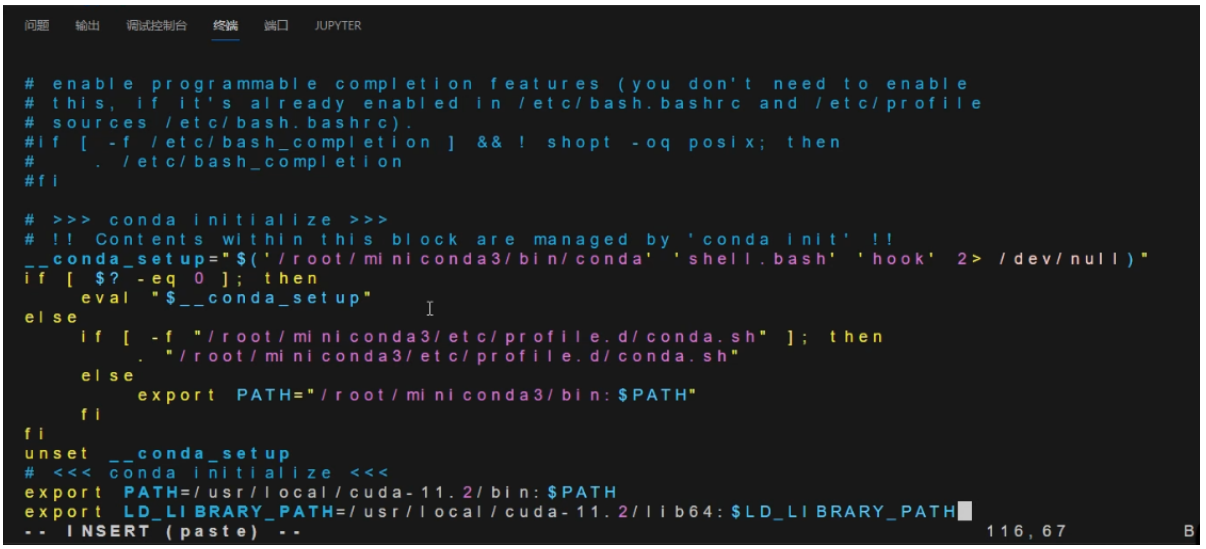


然后单击键盘上的“i”键，进入编辑模式，如下图所示。

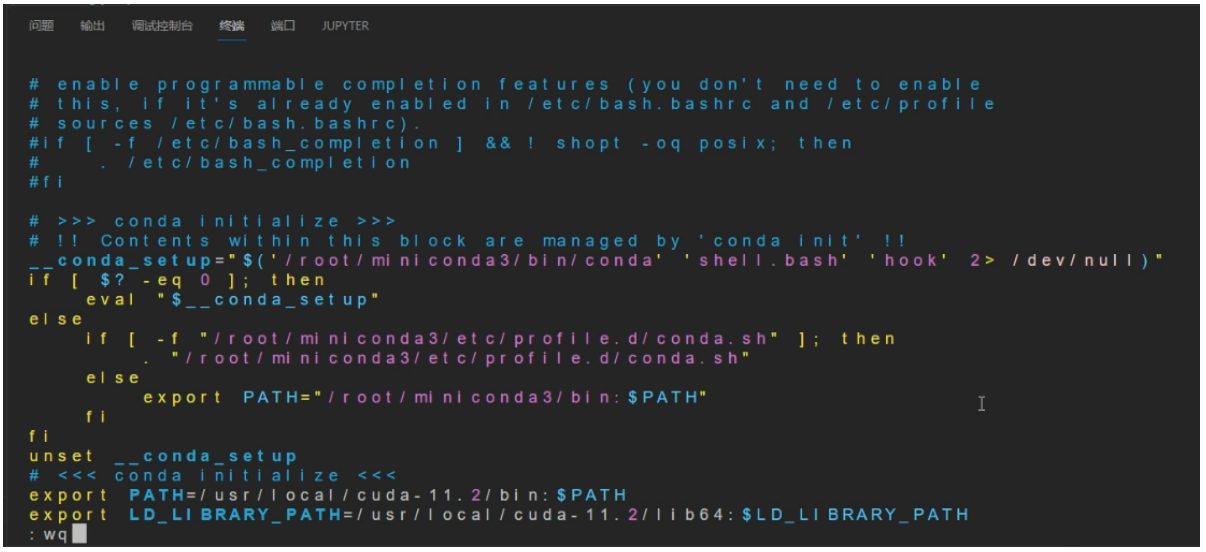


然后将下面这两行复制进去（先ctrl+c，然后ctrl+v复制到.bashrc文件中），复制完后如下图所示。

```
export PATH=/usr/local/cuda-11.2/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-11.2/lib64:$LD_LIBRARY_PATH
```



然后单击键盘上的“**esc**”键，然后输入“**:wq**”，如下图所示。

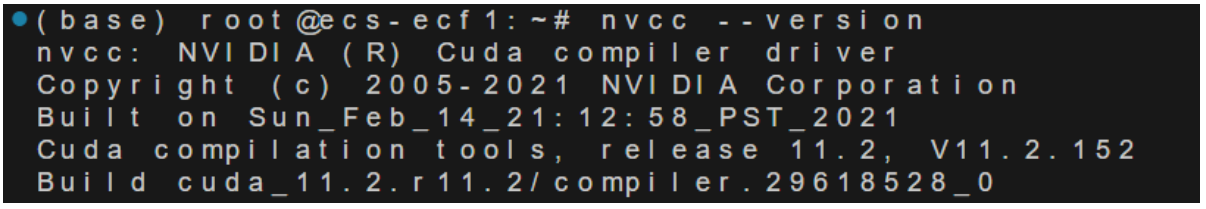


然后单击回车，即可退出.bashrc文件。

然后在终端里输入“**source ~/.bashrc**”命令，如下图所示，然后单击回车。



现在输入“**nvcc --version**”检查环境变量是否配置成功，此时应该就不会出现报错了，如下图所示。



然后在服务器终端输入“**sudo ldconfig /usr/local/cuda-11.2/lib64**”，回车。



再次运行finetune.ipynb中的第一个代码块尝试导入依赖包，此时报错消失，成功导入，如下图所示。



省流版

第1步：正常跟着课程配置环境，把该配置的都配置好。

第2步：在服务器终端在终端里输入“**source ~/.bashrc**”命令“**nvcc --version**”命令，如果出现“command not found”的报错，就接着执行下面的步骤；如果无报错，就不需要执行了。

第3步：在服务器终端中**输入**“**vim ~/.bashrc**”命令。

第4步：按住键盘上的“↓”键，将光标移动到最后一行；然后按住键盘上的“→”键，将光标移动到最后一个字符后面。

第5步：单击键盘上的“**i**”键。

第6步：将下面这两行复制进去：

```
export PATH=/usr/local/cuda-11.2/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-11.2/lib64:$LD_LIBRARY_PATH
```

第7步：单击键盘上的“**esc**”键，然后输入“**:wq**”，然后回车。

第8步：在终端里运行“**source ~/.bashrc**”命令。

第9步：在终端里运行“**nvcc --version**”检查环境变量是否配置成功。

第10步：在终端里运行“**sudo ldconfig /usr/local/cuda-11.2/lib64**”命令。

演示录屏

通过网盘分享的文件：修复录屏.mp4
链接: https://pan.baidu.com/s/1v2glvRJkmyEIe4SxsQBEzQ?pwd=paak 提取码: paak